

Specyfikacja Techniczna Aplikacji TipJar+

Wprowadzenie

Niniejszy dokument stanowi kompleksową specyfikację techniczną aplikacji TipJar+, pełniąc rolę kluczowego źródła wiedzy dla programistów, architektów i interesariuszy projektu. Jego celem jest szczegółowe przedstawienie architektury frontendowej i backendowej, implementacji kluczowych modułów funkcjonalnych, przepływów danych oraz technologii stanowiących fundament platformy. Dokument ten ma na celu zapewnienie spójnego zrozumienia systemu, co jest niezbędne dla jego dalszego rozwoju, utrzymania i skalowania.

1. Architektura Frontendowa: Fundament Aplikacji (UI Shell)

Moduł UI Shell stanowi strategiczny szkielet całej aplikacji frontendowej. Jego zadaniem jest nie tylko zdefiniowanie globalnego układu i nawigacji, ale również zapewnienie spójności wizualnej i technicznej poprzez scentralizowaną konfigurację stylów, typografii oraz kluczowych dostawców kontekstu (providers). Jest to fundament, na którym opierają się wszystkie kolejne, bardziej złożone moduły funkcjonalne.

1.1. Struktura i Komponenty Główne

Architektura UI Shell opiera się na kilku kluczowych komponentach, które razem tworzą spójne i reużywalne środowisko dla całej aplikacji.

- **RootLayout (`src/app/layout.tsx`)**: Główny komponent układu w architekturze Next.js App Router. Odpowiada za definicję bazowej struktury dokumentu HTML, w tym znaczników `<html>` i `<body>`. Integruje globalne arkusze stylów (`globals.css`), inicjalizuje zdefiniowane czcionki i, co kluczowe, jak wskazuje komentarz w kodzie źródłowym, „opakowujemy całą aplikację w Providers”.
- **Providers (`src/app/providers.tsx`)**: Pełni funkcję centralnego wrappera dla dostawców kontekstu. Jego głównym celem jest zapewnienie globalnej dostępności do kluczowych stanów i funkcjonalności, takich jak kontekst portfela blockchain (obsługiwany przez `wagmi`) oraz kontekst sesji uwierzytelnionego użytkownika (zarządzany przez NextAuth).

- **Navbar**: Stanowi stały element nawigacyjny, widoczny na górze interfejsu. Umożliwia użytkownikom łatwe poruszanie się po kluczowych sekcjach aplikacji, takich jak "Odkrywaj" (lista twórców) czy "Rejestracja", zapewniając spójne doświadczenie nawigacyjne.

1.2. Globalna Stylistyka i Typografia

System stylistyczny aplikacji został zdefiniowany centralnie w celu zapewnienia wizualnej spójności i zgodności z identyfikacją marki.

- **Typografia**: Aplikacja wykorzystuje dwie rodziny czcionek, aby rozróżnić tekst główny od elementów interfejsu, co poprawia czytelność i estetykę.
 - **Mukta Malar**: Służy jako główna czcionka dla treści tekstowych (font-sans). Dostępna jest w wariantach grubości **400** (regular) i **700** (bold).
 - **IBM Plex Sans**: Wykorzystywana jest dla elementów interfejsu użytkownika (UI), takich jak przyciski i nagłówki (font-ui). Dostępna w wariantach **400** (regular), **500** (medium) i **700** (bold).
- **Kolorystyka i Motyw**: Aplikacja oparta jest na ciemnym motywie ("dark mode"), co nadaje jej nowoczesny i elegancki wygląd. Paleta kolorów została precyzyjnie zdefiniowana w konfiguracji Tailwind CSS.
 - **Kolory brandowe**: **brand-dark** (#003737), **brand-gold** (#FFD700), **brand-purple** (#4D194D).
 - **Kolory tekstu**: **text-primary** (#DDE0DA) i **text-secondary** (#BCC1B6).
 - **Tło**: Tło aplikacji wykorzystuje charakterystyczny gradient, który dodaje głębi interfejsowi. Jego pełna definicja w pliku **globals.css** jest następująca:

Spójnie zaimplementowany UI Shell stanowi solidny fundament techniczny i wizualny, który gwarantuje jednolite doświadczenie użytkownika we wszystkich modułach aplikacji.

2. Zarządzanie Tożsamością i Cyklem Życia Użytkownika

Moduły uwierzytelniania i onboardingu są krytycznymi elementami platformy TipJar+. Razem tworzą one pierwsze, kluczowe doświadczenie użytkownika, definiując proces wejścia na platformę, a także zabezpieczają dostęp do jej

zasobów. Prawidłowe działanie tych systemów jest fundamentalne dla zaufania i zaangażowania użytkowników.

2.1. Moduł Uwierzytelniania (Auth)

Mechanizm uwierzytelniania został zaprojektowany w celu zapewnienia bezpiecznego i elastycznego dostępu do platformy, oferując wiele metod logowania i integrację z backendem.

- **Integracja z Backendem NestJS:** Frontend komunikuje się z dedykowanym modułem `Auth` w backendzie NestJS. Proces rejestracji odbywa się poprzez żądanie `POST /api/v1/auth/register`, a logowanie przez `POST /api/v1/auth/login`. Po pomyślnym uwierzytelnieniu backend zwraca tokeny JWT, które służą do autoryzacji kolejnych żądań. Sesja użytkownika jest weryfikowana na frontendzie poprzez odpytanie endpointu `/api/v1/auth/me`.
- **Strategie Uwierzytelniania:** Zaimplementowano trzy główne strategie logowania, aby sprostać różnym preferencjom użytkowników:
 - **E-mail/Hasło (Credentials):** Tradycyjna metoda logowania.
 - **OAuth:** Integracja z popularnymi dostawcami tożsamości, takimi jak **Google** i **Twitch**.
 - **"Sign-In with Ethereum" (SIWE):** Umożliwia logowanie za pomocą portfela kryptowalutowego. Proces ten polega na pobraniu jednorazowego numeru (`nonce`) z backendu, podpisaniu go przez portfel użytkownika i odesłaniu podpisu do weryfikacji.
- **Komponenty Interfejsu:** Głównym komponentem jest `LoginForm`, który służy jako centralny punkt do logowania i rejestracji. Inicjuje on sesję użytkownika w aplikacji za pośrednictwem biblioteki NextAuth.
- **Ochrona Tras (Route Guards):** Aplikacja implementuje mechanizm ochrony tras, który automatycznie przekierowuje niezalogowanych użytkowników próbujących uzyskać dostęp do chronionych sekcji (np. panelu twórcy) do strony logowania.

2.2. Proces Onboardingu Twórcy

Po pomyślnej rejestracji, nowi twórcy przechodzą przez wieloetapowy proces onboarding. Jego celem jest przeprowadzenie użytkownika przez konfigurację profilu w sposób przyjazny i ustrukturyzowany, co pozwala na

zebranie kluczowych danych i zapewnienie pozytywnego pierwszego doświadczenia.

Przepływ onboardingu został zsyntetyzowany z dwóch iteracji projektowych (v2.0 oraz wersji 5-etapowej), tworząc spójny proces:

1. **Tożsamość (Identity/Username):** Użytkownik ustawia swoją unikalną nazwę (`@handle`), która staje się częścią jego publicznego adresu URL profilu (np. `tipjar.plus/@nazwa`). Dostępność nazwy jest weryfikowana asynchronicznie.
2. **Uzupełnienie Profilu (Bio & Social):** W tym kroku twórca uzupełnia kluczowe dane publiczne, takie jak nazwa wyświetlana, biografia (bio), zdjęcie profilowe (avatar) oraz opcjonalnie cel finansowy, który będzie widoczny dla fanów.
3. **Portfel (Wallet):** Krok informacyjny, podczas którego twórca dowiaduje się o swoim automatycznie utworzonym portfelu USDC lub ma możliwość podłączenia własnego.
4. **Powiadomienia (Notifications):** Konfiguracja preferencji dotyczących powiadomień, np. zgoda na otrzymywanie e-maili o nowych napiwkach.
5. **Zgody i Zakończenie (Consents & Done):** Ostatni etap to zebranie zgód (np. regulamin) oraz podsumowanie procesu. Po zatwierdzeniu, w backendzie ustawiana jest flaga `hasCompletedOnboarding = true` , a użytkownik jest przekierowywany do swojego panelu.

Za realizację tego procesu odpowiada komponent `CreatorOnboardingWizard` wraz z dedykowanymi podkomponentami dla każdego kroku. Kluczowym elementem jest "guard onboarding", który wymusza ukończenie procesu. Mechanizm ten działa na dwóch poziomach: po stronie serwera (middleware w Next.js) przechwytyje żądania użytkownika z `hasCompletedOnboarding=false` i przekierowuje go do `/onboarding` , a po stronie klienta (hook w komponentach) zapewnia dodatkową weryfikację i przekierowanie, gwarantując, że twórca nie pominie konfiguracji profilu.

Po pomyślnym uwierzytelnieniu i onboarding, użytkownicy uzyskują dostęp do kluczowych funkcji platformy, w tym do pełnej integracji z portfelem Circle, która stanowi rdzeń systemu finansowego.

3. Integracja z Circle API: Rdzeń Systemu Płatności

Integracja z Circle API odgrywa fundamentalną rolę w modelu biznesowym TipJar+, stanowiąc technologiczny kręgosłup dla wszystkich operacji finansowych na platformie. Umożliwia ona tworzenie i zarządzanie portfelami kustodialnymi w standardzie USDC, co jest podstawą dla systemu napiwków i wypłat.

3.1. Architektura Backendowa (NestJS)

Backend aplikacji, zbudowany w oparciu o framework NestJS, jest odpowiedzialny za całą komunikację z Circle API, co zapewnia bezpieczeństwo i centralizację logiki biznesowej.

- **Moduł `CircleService`**: Jest to dedykowany serwis, którego główną odpowiedzialnością jest tworzenie i obsługa portfeli kustodialnych w modelu Developer Controlled Wallets (DCW). Abstrakcjonuje on złożoność wywołań API Circle, udostępniając prosty interfejs dla reszty aplikacji.
- **Tworzenie Portfeli Użytkowników**: Dla każdego nowo zarejestrowanego twórcy system automatycznie tworzy dedykowany portfel USDC. Proces ten jest inicjowany przez backend, a uzyskane identyfikatory (`circleWalletId`) oraz adres portfela (`mainWalletAddress`) są bezpiecznie zapisywane w bazie danych aplikacji i powiązane z kontem użytkownika.
- **Obsługa Operacji Finansowych**: Moduł `CircleService` jest wykorzystywany do orkiestracji wszystkich operacji finansowych. Obsługuje on zarówno **wypłaty (Payouts)** środków z portfeli twórców na zewnętrzne adresy, jak i wewnętrzne **transfery napiwków (Tips)** pomiędzy portfelami fanów i twórców, które odbywają się w ramach ekosystemu Circle.

3.2. Prezentacja Danych Portfela na Frontendzie

Informacje finansowe pochodzące z Circle API są w przejrzysty sposób udostępniane użytkownikom w interfejsie aplikacji, przy jednoczesnym ukryciu złożoności technologicznej.

- Po zalogowaniu, aplikacja frontendowa pobiera z backendu aktualne informacje o saldzie portfela USDC przypisanego do użytkownika.
- Dane te są prezentowane w dedykowanych panelach:
 - **Studio Twórcy**: Wyświetla saldo dostępne do wypłaty, historię transakcji oraz interfejs do zlecenia wypłat.

- **Dashboard Fana:** Pokazuje balans środków dostępnych do wysyłania napiwków oraz historię wsparcia.
- Dla użytkownika końcowego interakcja z portfelem przypomina korzystanie z tradycyjnej aplikacji finansowej. Ta abstrakcja jest podstawowym założeniem architektonicznym, zaprojektowanym w celu drastycznego obniżenia bariery wejścia dla użytkowników niezaznajomionych z technologią krypto i pozycjonowania TipJar+ jako przyjaznej aplikacji fintechowej, a nie niszowego narzędzia Web3.

Solidna infrastruktura płatnicza oparta na Circle API stanowi fundament dla kluczowych modułów funkcjonalnych, które realizują podstawowe założenia biznesowe platformy.

4. Kluczowe Moduły Funkcjonalne

Na solidnych fundamentach architektury, uwierzytelniania i integracji płatności zbudowano kluczowe moduły, które realizują główne funkcje platformy. Moduły te tworzą spójny ekosystem umożliwiający interakcję między twórcami a ich fanami, od odkrywania profili po zarządzanie finansami.

4.1. Moduł Odkrywania Twórców (Explorer)

Celem modułu "Explorer" jest umożliwienie użytkownikom przeglądania, wyszukiwania i odkrywania twórców zarejestrowanych na platformie. Jego rozwój przebiegał iteracyjnie, co pozwoliło na stopniowe wdrażanie funkcjonalności.

- **v1.1:** Stworzono podstawowy layout strony z wykorzystaniem komponentów `CreatorsPage` i `CreatorCard`. Na tym etapie dane były statyczne (mockowane), co pozwoliło na zbudowanie szkieletu interfejsu.
- **v1.2:** Zintegrowano moduł z backendem poprzez endpoint `GET /api/v1/creators`, co umożliwiło wyświetlanie rzeczywistych danych. Dodano również komponent `SearchBar` do wyszukiwania twórców.
- **v1.3:** Wprowadzono paginację (lub "infinite scroll") w celu optymalizacji ładowania dużej liczby profili. Udoskonalono również wygląd komponentu `CreatorCard`.
- **v1.4:** Sfinalizowano moduł poprzez stworzenie dynamicznych stron profili twórców, dostępnych pod unikalnymi adresami URL w formacie

`app/@[handle]/page.tsx` . Zapewniono płynne przejście od listy w Explorerze do szczegółowego profilu.

Komponent `CreatorProfilePage` stanowi zwieńczenie tego modułu. Prezentuje on pełne informacje o twórcy, takie jak biografia, cele finansowe i linki społecznościowe, a także zawiera formularz umożliwiający wysłanie napiwku. Dane dla tej strony są pobierane z dedykowanego endpointu `GET /api/v1/creator/{handle}` .

4.2. Panel Twórcy (Studio)

Panel "Studio" to dedykowane centrum zarządzania dla twórców, zapewniające im pełną kontrolę nad swoim profilem i finansami.

- **Struktura i Nawigacja:** Układ panelu opiera się na bocznym menu (sidebar) oraz nagłówku (header). Nawigacja umożliwia dostęp do kluczowych sekcji, takich jak: Strona główna/Statystyki, Profil, Wyплаты, Cel i Ustawienia.
- **Funkcjonalności:** Panel oferuje szeroki zakres możliwości:
 - **Pulpit Główny:** Prezentuje kluczowe statystyki, w tym sumę otrzymanych środków USDC oraz liczbę napiwków, dając twórcy szybki wgląd w jego wyniki.
 - **Personalizacja Profilu:** Umożliwia edycję publicznych danych, takich jak avatar, nazwa wyświetlana, biografia i linki społecznościowe. Formularze komunikują się z endpointem `PATCH/PUT /api/v1/users/profile/update` , który obsługuje zarówno częściowe (`PATCH`), jak i pełne (`PUT`) aktualizacje danych profilowych.
 - **Zarządzanie Finansami:** Daje możliwość zlecenia wypłaty zgromadzonych środków USDC na zewnętrzny portfel kryptowalutowy lub konto bankowe za pośrednictwem endpointu `POST /api/v1/payouts` .

4.3. Panel Użytkownika (Dashboard)

Panel "Dashboard" to przestrzeń przeznaczona dla zalogowanych fanów, skupiająca funkcje związane ze wspieraniem twórców.

- **Tablica Wsparcia:** Główny widok panelu, prezentujący feed aktywności, który zawiera przede wszystkim historię wysłanych napiwków.
- **Portfel Fana:** Umożliwia fanowi zarządzanie własnym saldem USDC, w tym dodawanie środków w celu przyszłego wspierania twórców oraz ich

ewentualną wypłatę.

- **Obserwowani Twórcy:** Sekcja, w której fan może przeglądać listę twórców, których aktywnie wspiera lub obserwuje, co ułatwia ponowne interakcje.

Te trzy moduły – Explorer, Studio i Dashboard – tworzą kompletny ekosystem interakcji, realizując podstawowe założenia biznesowe aplikacji i zapewniając płynne doświadczenie zarówno dla twórców, jak i ich fanów.

5. Współdzielone Komponenty UI i Strategia Testowania

Strategiczne wdrożenie reużywalnych komponentów UI jest fundamentem dla utrzymania spójności w ramach Design Systemu, przyspieszenia procesu deweloperskiego i redukcji długu technicznego. Równolegle, rygorystyczna strategia zautomatyzowanych testów gwarantuje stabilność, niezawodność i de-ryzykuje przyszły rozwój poprzez wczesne wykrywanie regresji.

5.1. Kluczowe Reużywalne Komponenty

W całej aplikacji wykorzystano zestaw współdzielonych komponentów, które standaryzują wygląd i zachowanie kluczowych elementów interfejsu.

Komponent	Opis i Zastosowanie
Avatar	Komponent <code>Avatar.tsx</code> renderuje obraz profilowy użytkownika. W przypadku braku obrazu (<code>src</code>), automatycznie wyświetla estetyczny placeholder z inicjałami użytkownika. Jest szeroko stosowany w nagłówku, na kartach twórców (<code>CreatorCard</code>) oraz na stronach profili.
Card	<code>Card.tsx</code> to stylowy kontener do opakowywania treści. Zapewnia spójny wygląd dla elementów takich jak kafelki twórców w module Explorer oraz sekcje statystyk na pulpicie głównym panelu twórcy, standaryzując cienie, tła i zaokrąglenia.
TipForm	<code>TipForm.tsx</code> jest kluczowym komponentem do wysyłania napiwków. Składa się z pola na kwotę, opcjonalnego pola na wiadomość oraz przycisku akcji (CTA). Zarządza swoim wewnętrznym stanem, np. podczas wysyłania żądania do API <code>/api/v1/tips</code> blokuje przycisk i zmienia jego etykietę na "Wysyłanie...".
Header	<code>Header.tsx</code> to komponent nagłówka, który dynamicznie dostosowuje swój wygląd. Renderuje inne elementy w zależności od kontekstu, np. przyciski logowania dla użytkowników publicznych lub menu profilowe

z awatarem dla zalogowanych, zapewniając spójność nawigacji globalnej.
--

5.2. Testy End-to-End (Playwright)

W celu zapewnienia najwyższej jakości aplikacji, wdrożono strategię testów end-to-end (E2E) z wykorzystaniem frameworka Playwright. Testy te symulują rzeczywiste interakcje użytkownika z aplikacją, weryfikując kluczowe ścieżki i funkcjonalności od początku do końca.

Zautomatyzowano następujące scenariusze testowe:

1. **Pełna ścieżka twórcy:** Test obejmuje kompleksowy przepływ: tworzy unikalny email, przechodzi przez formularz rejestracji, pomyślnie kończy wszystkie kroki onboarding, a następnie loguje się jako osobny testowy użytkownik (fan), odnajduje profil nowo utworzonego twórcy, wysyła napiwek i weryfikuje poprawność zmiany salda.
2. **Działanie Guardów i Przekierowań:** Scenariusze weryfikują krytyczne mechanizmy ochrony tras:
 - Próba dostępu do `/dashboard` przez niezalogowanego użytkownika skutkuje natychmiastowym przekierowaniem do `/login`.
 - Próba dostępu do `/login` przez zalogowanego użytkownika skutkuje przekierowaniem do `/dashboard`.
 - Użytkownik z flagą `hasCompletedOnboarding=false` jest zawsze przekierowywany z powrotem do `/onboarding`, niezależnie od chronionej trasy, do której próbuje uzyskać dostęp.
3. **Poprawność Interfejsu:** Testy sprawdzają poprawność wyświetlania kluczowych elementów UI na poziomie piksela. Weryfikują m.in. dokładną treść placeholderów (np. `"Wprowadź email"`), treść komunikatów o błędach walidacji oraz poprawne pojawianie się powiadomień typu toast/modal po wykonaniu akcji.

Przedstawiona w niniejszym dokumencie architektura, szczegółowo opisane moduły funkcjonalne oraz zdefiniowane procesy testowania tworzą razem solidną i skalowalną podstawę dla dalszego rozwoju platformy TipJar+.